

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 2

Software Design

Author: Maria Moruțan

Group: 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

April 2026

Contents

1	Problem definition	2
1.1	Users and roles	2
2	Architecture evolution	2
2.1	Service-based modular architecture	2
2.2	Event-driven communication	3
3	System diagrams	4
3.1	Use case diagram	4
3.2	Entity-relationship diagram	5
3.3	Class diagram	6
3.4	Sequence diagram	7
4	Functional extensions	7
4.1	Event system and notifications	7
4.2	Export functionality	8
5	Design patterns	8
6	Implementation details	8
6.1	Event system implementation	8
6.2	Notification service	9
6.3	Export system implementation	9
7	User interface updates	9
8	Testing and validation	10

1 Problem definition

This assignment extends the Pet Management System developed in Assignment 1 by introducing a more modular, service-oriented architecture and event-driven communication between system components. The domain remains unchanged, but the internal structure of the application has been refactored to improve scalability, maintainability, and separation of concerns.

The main objective is to decouple responsibilities into independent services and enable communication between them using events. Additionally, new functionalities such as email notifications and data export in multiple formats are introduced, while making use of fundamental design patterns such as Observer, Strategy and Singleton.

1.1 Users and roles

The system continues to support the same roles defined in Assignment 1:

- **Visitor**– can view pets and items but cannot modify data.
- **Administrator**- has full access to all system functionalities, including CRUD operations on all entities.
- **Seller**– can add pets and items but cannot update or delete them.

The role-based access control logic remains unchanged but is now better integrated within the service layer. Thus, any type of user can export selected data in any format wanted and an authenticated user(e.g., for users that have the Administrator role) will receive email notifications upon a CRUD operation being performed in the system.

2 Architecture evolution

2.1 Service-based modular architecture

The application evolves from a simple layered architecture into a modular service-based architecture. Although all components are still part of the same application, responsibilities are clearly separated into distinct services:

- **PetServiceClass / ItemServiceClass (Resource Management Services):** Handle CRUD operations, filtering, sorting, and business logic related to domain entities.
- **UserService (User Management Service):** Handles authentication and user-related operations.
- **NotificationService:** Listens to system events and sends email notifications.
- **ExportService:** Responsible for exporting domain data in multiple formats.
- **DTO Layer (Data Transfer Objects):** Used to transform domain entities into simplified representations (e.g., `PetDTO`, `ItemDTO`) for export. This ensures that only relevant fields are exposed and allows customization of output formats without affecting the core domain model.

2.2 Event-driven communication

Communication between services is partially event-driven. Instead of tightly coupling services, an event mechanism is introduced using the Observer design pattern.

A central **EventManager** acts as a dispatcher:

- Services emit events when important actions occur.
- Other services subscribe and react to these events.

For example, when a Pet or Item is created, updated, or deleted, a **ResourceEvent** is generated and sent to all subscribed listeners.

3 System diagrams

3.1 Use case diagram

The use case diagram illustrates the interactions between system actors and the main functionalities provided by the application. The three roles (Visitor, Administrator, Seller) interact differently with the system based on their permissions.

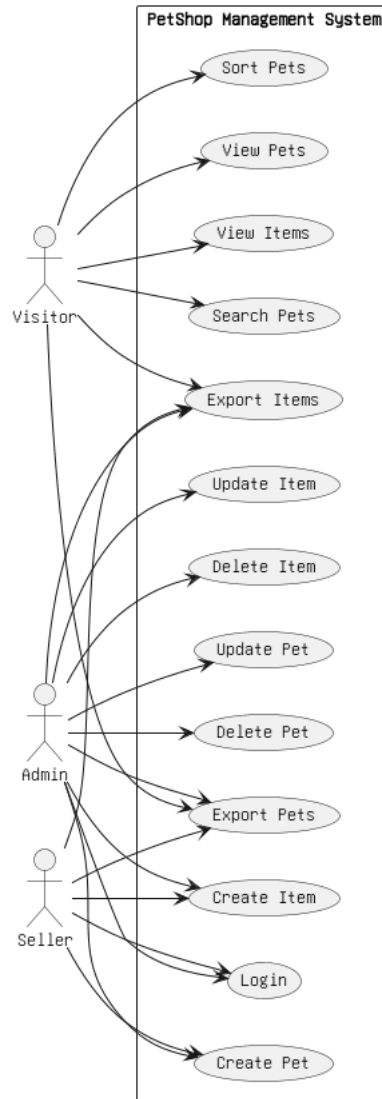


Figure 1: Use-case diagram for the Pet Management System

3.2 Entity-relationship diagram

The entity-relationship diagram represents the database structure of the system. The main entities are User, Pet, and Item, with a one-to-many relationship between Pet and Item.

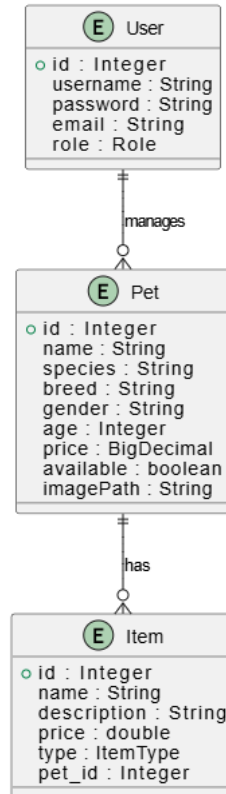


Figure 2: Entity-relationship diagram

3.3 Class diagram

The class diagram illustrates the core components of the system, including services, entities, event handling, and export strategies. It highlights the modular architecture and the use of design patterns.

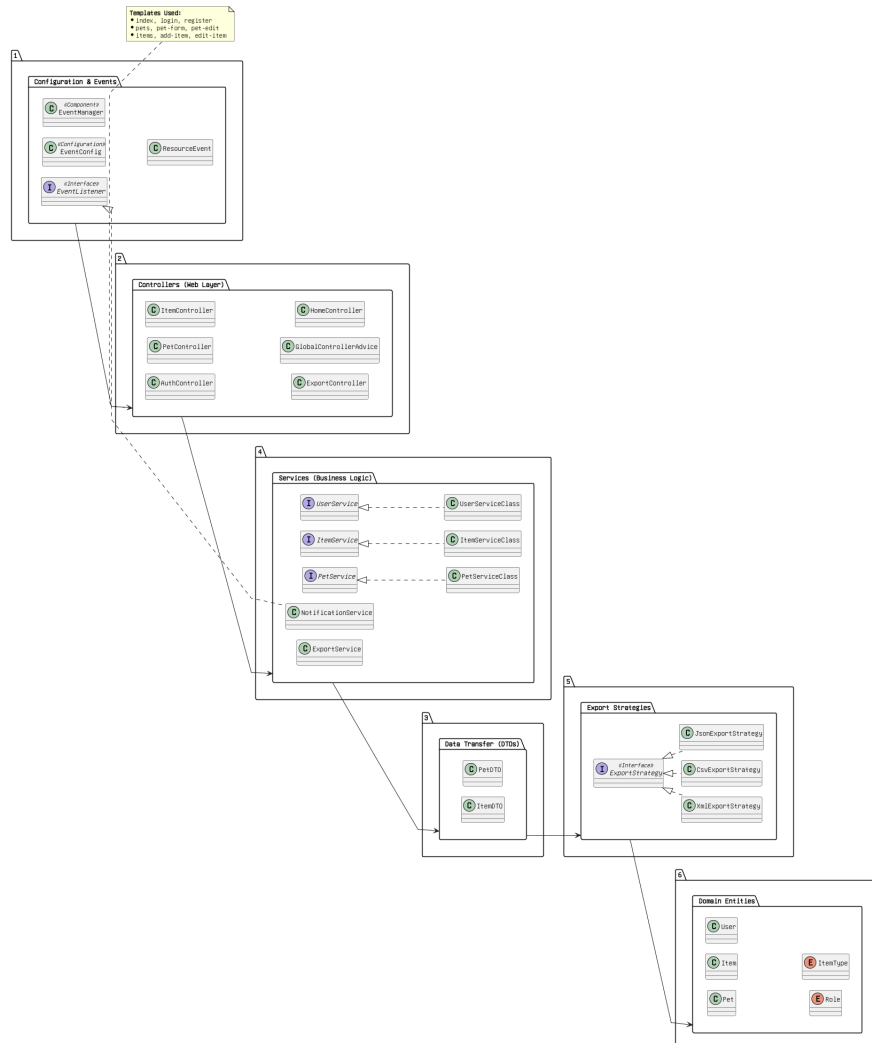


Figure 3: Class diagram of the system

3.4 Sequence diagram

The sequence diagram illustrates the workflow of creating a new Pet and triggering an event that results in an email notification being sent.

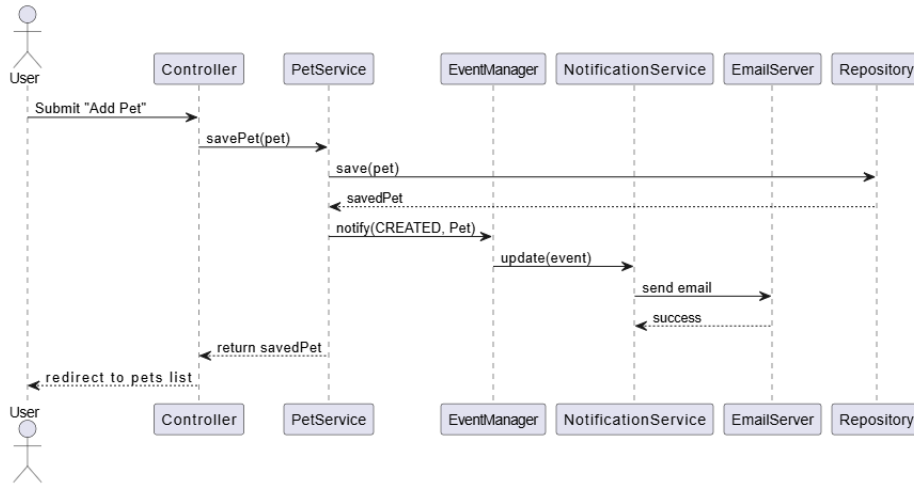


Figure 4: Sequence diagram for pet creation and notification

4 Functional extensions

4.1 Event system and notifications

The application now supports event-driven behavior. Resource management services (e.g., PetService, ItemService) emit events when:

- an entity is created
- an entity is updated
- an entity is deleted

These events are handled by the `NotificationService`, which:

- listens to events via the `EventManager`
- retrieves the currently authenticated user from the session
- sends an email notification describing the performed action

Example notification message:

"Pet was UPDATED" or "Item was CREATED"

Email sending is implemented using Gmail SMTP configuration with application password.

4.2 Export functionality

A new export feature allows users to download domain data dynamically in multiple formats:

- JSON
- CSV
- XML

The export system supports:

- exporting all entities
- exporting filtered data (e.g., pets filtered by name)
- exporting sorted data (e.g., by price or age)

Export is triggered directly from the user interface via dedicated buttons.

5 Design patterns

Several design patterns were introduced to improve modularity and flexibility:

- **Singleton Pattern:**
 - Used for managing shared components such as the database connection and event manager.
 - Spring annotations such as `@Service`, `@Controller` are used in classes, for example in `NotificationService` and `ExportService`, which help injecting the objects created into Spring containers.
- **Observer Pattern:**
 - Implemented via `EventManager` and `EventListener`.
 - Enables event-driven communication between services.
- **Strategy Pattern:**
 - Used for implementing multiple export formats.
 - Each format (JSON, CSV, XML) is implemented as a separate strategy.

6 Implementation details

6.1 Event system implementation

The event system is built using the following components:

- **ResourceEvent** – encapsulates event type, entity type, and payload
- **EventManager** – manages subscribers and dispatches events
- **EventListener** – interface implemented by services that react to events

Services emit events using:

```
eventManager.notify(new ResourceEvent("CREATED", "Pet", pet));
```

6.2 Notification service

The **NotificationService** subscribes to events and sends email notifications to the currently logged-in user. It retrieves user information from the HTTP session and constructs a message based on the event type.

6.3 Export system implementation

The export functionality is implemented using the Strategy pattern:

- **ExportStrategy** – interface defining export behavior
- **JsonExportStrategy**, **CsvExportStrategy**, **XmlExportStrategy** – concrete implementations
- **ExportContext** – selects the appropriate strategy
- **ExportService** – exposes a unified export method to controllers

The controller layer calls:

```
exportService.export(data, format);
```

7 User interface updates

The UI was extended with export controls:

- Buttons for exporting pets and items
- Dropdowns for selecting export format
- Integration with filtering and sorting inputs

The export functionality is triggered through HTTP GET requests and returns dynamically generated content.

8 Testing and validation

The system was tested to ensure:

- Events are correctly triggered on create, update, and delete operations
- Notification service receives and processes events
- Emails are sent correctly using SMTP configuration
- Export functionality works for all formats (JSON, CSV, XML)
- Filtering and sorting are correctly applied before export

Testing confirmed that all new requirements are successfully implemented and integrated with the existing system.